# 09 PIPES IN NESTJS (ParseIntPipe AND FRIENDS)

A Pipe is a class that runs BEFORE a route handler's arguments reach the method body. It does one (or both) of two jobs:

1. TRANSFORMATION — convert input data to the desired type/shape.
2. VALIDATION — check the input is valid, and throw an error (usually a 400 Bad Request) if it isn't.

```
Request arrives -> Pipe runs on the argument -> handler method runs
                   (transform/validate)         (gets clean, typed data)
```

## WHY PIPES EXIST — THE PROBLEM THEY SOLVE

Everything coming from an HTTP request — route params, query strings, request bodies — arrives as a STRING (or raw JSON), never as a proper typed value. Without a pipe:

```
@Get(':id')
getProductById(@Param('id') id: string) {   // id is ALWAYS a string
  const product = this.products.find(p => p.id === id);  // BUG!
  // p.id is a number, id here is a string — "1" !== 1 in strict
  // comparisons, or you'd need to manually parseInt() every time.
}
```

Pipes remove that manual, repetitive conversion/validation code from every handler.

## REAL EXAMPLE FROM THIS PROJECT

```
// user.controller.ts
@Get('products/:id')                                    // GET /user/products/1
getProductById(@Param('id', ParseIntPipe) id: number) {
  return this.userService.getProductById(id);
}
```

- @Param('id', ParseIntPipe) means: "take the ':id' URL segment (a string like '1'), run it through ParseIntPipe, and give me back a real number."
- Inside the method, `id` is typed as `number` — genuinely, not just a TypeScript label. ParseIntPipe actually converts the value at runtime.
- If someone requests GET /user/products/abc (not a number), ParseIntPipe throws automatically — Nest turns that into a 400 Bad Request response, WITHOUT you writing any if-checks yourself:

```
{
  "statusCode": 400,
  "message": "Validation failed (numeric string is expected)",
  "error": "Bad Request"
}
```

## BUILT-IN PIPES NEST PROVIDES

| | |
|---|---|
| **ParseIntPipe** | String → integer. Throws 400 if not numeric. |
| **ParseFloatPipe** | String → float. |
| **ParseBoolPipe** | String ('true'/'false') → boolean. |
| **ParseArrayPipe** | String/CSV → array. |
| **ParseUUIDPipe** | Validates the string is a proper UUID format. |
| **ParseEnumPipe** | Validates the value is one of a given enum's values. |
| **DefaultValuePipe** | Supplies a default if the value is undefined (commonly paired with @Query, e.g. pagination): |

```
  @Get()
  findAll(@Query('page', new DefaultValuePipe(1), ParseIntPipe) page: number) {
    // ?page not given -> defaults to 1, still converted to a number
  }

ValidationPipe       The big one — validates an entire @Body() object
                     against a DTO class decorated with class-validator
                     rules (e.g. @IsString(), @IsInt(), @Min(0)).
                     Usually applied globally in main.ts:

  app.useGlobalPipes(new ValidationPipe());
```

## WHERE PIPES CAN BE APPLIED

1. Parameter-level (most common, most targeted): @Param('id', ParseIntPipe) id: number
2. Method-level (applies to ALL params of one handler): @UsePipes(ValidationPipe) @Post() create(@Body() dto: CreateProductDto) { ... }
3. Controller-level (applies to every route in the controller): @UsePipes(ValidationPipe) @Controller('products') export class ProductsController { ... }
4. Global (applies app-wide, set once in main.ts): app.useGlobalPipes(new ValidationPipe());

## CUSTOM PIPES (WHEN THE BUILT-INS AREN'T ENOUGH)

You can write your own by implementing PipeTransform:

```
import { PipeTransform, Injectable, BadRequestException } from '@nestjs/common';

@Injectable()
export class PositiveIntPipe implements PipeTransform {
  transform(value: string): number {
    const val = parseInt(value, 10);
    if (isNaN(val) || val <= 0) {
      throw new BadRequestException('Value must be a positive integer');
    }
    return val;
```

```
  }
}
```

**// usage:**

```
@Get(':id')
findOne(@Param('id', PositiveIntPipe) id: number) { ... }
```

Use this when validation rules are specific to your domain (e.g. "must be a positive integer," not just "must be an integer").

## PIPES vs GUARDS vs INTERCEPTORS (QUICK DISTINCTION)

| | |
|---|---|
| **Pipe** | — transforms/validates ONE argument, right before the handler runs. Scope: input data shape/validity. |
| **Guard** | — decides IF the request is allowed to reach the handler at all (auth/roles). Scope: access control. |
| **Interceptor** | — wraps the whole call, can run logic before AND after (logging, response transforming, timing). Scope: cross- cutting behavior around the entire request/response. |

They all run in a defined order: Middleware → Guards → Interceptors (before) → Pipes → Route Handler → Interceptors (after) → Filters (if an error was thrown).

### ★ RULE OF THUMB

Any time a route argument needs to be converted to a different type (string → number/boolean/UUID) or validated before your handler logic runs, reach for a pipe instead of writing manual parseInt()/if-checks inside the method body. It keeps handlers focused on business logic, not input plumbing.

See also: **theory/02-controller.txt**, **theory/06-decorators.txt**